

SOFTWARE TESTING – CST458

Module - 4 (Unit Testing - Black Box Approaches)

SYLLABUS : Module - 4

- Domain Testing / Input Space Partitioning - Partitions of a set. Input domain modelling - Interface-based approach, Functionality-based approach. Identifying values.
- Multiple partitions of the input domain - All Combinations Coverage (ACoC), Each Choice Coverage (ECC), Pair-wise Coverage, T-wise Coverage, Base Choice Coverage, Multiple Base Choices Coverage. TriTyp example.
- Functional Testing - Functional Testing Concepts of Howden. Functional testing - Important Steps.
- Types of Functional testing - Equivalence Class Partitioning, Boundary Value Analysis, Decision Tables, Random Testing.
- Case Study - Black Box testing approaches using JUnit.

What is Domain Testing?

- What is DOMAIN ?
- Domain Testing is a Software Testing process
- The application/Software is tested by giving a minimum number of inputs and evaluating its appropriate outputs
- It verifies that the system should not accept inputs and/or conditions outside the specified or valid range
- Domain might involve testing of any one input variable or combination of input variables

Domain Testing

- A domain is divided into sub-domains (equivalence classes) and then test using values from each sub-domain
- Example:
 - ✓ A website (domain) has been given for testing, we will be dividing the website into small portions (subdomain) for the ease of testing
- Advantages:
 - to check whether the software application accepts inputs within the acceptable range
 - delivers required output
 - to ensure that the system does not accept invalid and out of range input values

INPUT DOMAIN MODELING (IDM)

- Three Steps:

1. To identify testable functions
2. to identify all of the parameters that can affect the behavior of a given testable function
3. To model the input domain articulated in the prior step

- Two approaches to input domain modeling:

- A. **Interface-based approach** : Develops characteristics directly from input parameters to the program under test
- B. **Functionality-based approach** : Develops characteristics from a functional view of the program under test

INPUT DOMAIN MODELING (IDM)

Interface-based approach :

- The interface-based approach considers each parameter in isolation.
- It identifies the type and range of each parameter, and then generates test cases that cover all possible values within the range.
- This approach is simple and easy to automate, but it can be incomplete if it does not take into account the relationships between parameters.

INPUT DOMAIN MODELING (IDM)

Interface-based approach :

For example, consider a function that takes two parameters: a number and a string. The interface-based approach would generate test cases that cover all possible values for each parameter, such as:

Code snippet

Number: 1, 2, 3, 4, 5

String: "a", "b", "c", "d", "e"

However, this approach would not generate test cases that test the interaction between the two parameters, such as:

Code snippet

Number: 0

String: ""

INPUT DOMAIN MODELING (IDM)

Functionality-based approach :

- The functionality-based approach considers the functional or behavioral view of the program under test.
- It identifies the different ways in which the program can be used, and then generates test cases that exercise each of these ways.
- This approach is more comprehensive than the interface-based approach, but it can be more difficult to automate.

INPUT DOMAIN MODELING (IDM)

For example, consider a function that takes a number and returns the square of that number. The functionality-based approach would generate test cases that cover all possible ways in which the function can be used, such as:

Code snippet

Number: 1, 2, 3, 4, 5

Expected output: 1, 4, 9, 16, 25

This approach would also generate test cases that test the edge cases, such as:

Code snippet

Number: 0

Expected output: 0

INPUT DOMAIN MODELING (IDM)

Which approach to use?

- The best approach to use depends on the specific software being tested.
- The interface- based approach is a good choice for software with a simple interface and well-defined parameters.
- The functionality-based approach is a good choice for software with a complex interface or parameters that can be used in multiple ways.

FUNCTIONAL TESTING

- Its a type of black-box testing
- It checks if each feature works as per the software requirements
- Each function is compared to corresponding requirement to check if the output is consistent with the end user's expectations
- Internal program structure is NOT considered
- Three key elements of a function are its input, output, and transformation (expected) of input to output
- Not concerned with details of how an input is transformed into an output, but a program is treated as a function in the general sense

FUNCTIONAL TESTING – 4 Concepts

Four key concepts in functional testing are:

1. Identify the domain of each input and each output variable
2. Select values from the data domain of each variable having important properties
3. Consider combinations of special values from different input domains to design test cases
4. Consider input values such that the program under test produces special values from the domains of the output variables

FUNCTIONAL TESTING – Types

- I. Equivalence Class Partitioning
- II. Boundary Value Analysis
- III. Decision Tables
- IV. Random Testing

Equivalence Class Partitioning

- Input data is divided into partitions of valid and invalid values
- All partitions must exhibit the same behavior:
 - ❖ If a condition of one partition is true, then the condition of another equal partition must also be true
 - ❖ If a condition of one partition is false, then the condition of another equal partition must also be false
- Test cases should be designed to cover each partition at least once
- partitions are derived from requirements and specifications of the software
- Advantage :
 - ❑ helps to reduce the time of testing
- Applicable at all levels of the testing process

Equivalence Class Partitioning - Example

Enter 10 Digit Mobile number

INVALID 1 Test case	INVALID 2 Test case	VALID 3 Test case	VALID
DIGITS >=11	DIGITS<=9	DIGITS = 10	DIGITS =10
93847262219	984543985	9991456234	9893451483

Boundary Value Analysis

- To test the boundary values of valid and invalid partitions
- To select test data near the boundary of a data domain
- Edge of the equivalence partition is more likely to be incorrect than the behavior within the partition
- Boundaries are an area where testing is likely to yield defects
- It checks for the input values near the boundary that have a higher chance of error
- Data both within and outside an equivalence class are selected
- Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition.

Decision Tables

- Decision tables are used in various engineering fields
- to represent complex logical relationships
- Its a very effective tool in testing the software and its requirements management
- The output may be dependent on many input conditions and decision tables give a tabular view of various combinations of input conditions and these conditions are in the form of True(T) and False(F)
- Provides a set of conditions and its corresponding actions required in the testing.

Decision Tables

- It comprises a set of conditions (or, causes) and a set of effects (or, results) arranged in the form of a column on the left of the table
- In the second column, next to each condition, we have its possible values: Yes (Y), No (N), and Don't Care ("-")
- To the right of the "Values" column, we have a set of rules. For each combination of the three conditions {C1,C2,C3}, there exists a rule from the set {R1,R2, ...,R8}

Conditions	Values	Rules or Combinations							
		<i>R</i> ₁	<i>R</i> ₂	<i>R</i> ₃	<i>R</i> ₄	<i>R</i> ₅	<i>R</i> ₆	<i>R</i> ₇	<i>R</i> ₈
<i>C</i> ₁	Y, N,—	Y	Y	Y	Y	N	N	N	N
<i>C</i> ₂	Y, N,—	Y	Y	N	N	Y	Y	N	N
<i>C</i> ₃	Y, N,—	Y	N	Y	N	Y	N	Y	N
Effects									
<i>E</i> ₁		1		2	1				
<i>E</i> ₂			2	1			2	1	
<i>E</i> ₃		2	1	3		1	1		
Checksum	8	1	1	1	1	1	1	1	1

Decision Tables

- Each rule comprises a Yes (Y), No (N), or Don't Care ("-") response, and contains an associated list of effects {E1,E2,E3}
- For each relevant effect, an effect sequence number specifies the order in which the effect should be carried out, if the associated set of conditions are satisfied
- The "Checksum" is used for verification of the combinations, the decision table represent
- Each rule of a decision table represents a test case

Conditions	Values	Rules or Combinations							
		<i>R</i> ₁	<i>R</i> ₂	<i>R</i> ₃	<i>R</i> ₄	<i>R</i> ₅	<i>R</i> ₆	<i>R</i> ₇	<i>R</i> ₈
<i>C</i> ₁	Y, N,—	Y	Y	Y	Y	N	N	N	N
<i>C</i> ₂	Y, N,—	Y	Y	N	N	Y	Y	N	N
<i>C</i> ₃	Y, N,—	Y	N	Y	N	Y	N	Y	N
Effects									
<i>E</i> ₁		1		2	1				
<i>E</i> ₂			2	1			2	1	
<i>E</i> ₃		2	1	3		1	1		
Checksum	8	1	1	1	1	1	1	1	1

Random Testing

- The system is tested with the help of generating random and independent inputs and test cases
- It is a black box assessment outline technique
- Tests are being chosen randomly and the results are being compared by some software identification to check whether the output is correct or incorrect.
- Steps in Random Testing:
 - ❑ Step-1: Identify Input domain
 - ❑ Step-2: Select test inputs independently/randomly from the input domain
 - ❑ Step-3: Test the system on these inputs and form a random test set
 - ❑ Step-4: Compare the result with system specification
 - ❑ Step-5: If the Report fails then take necessary action.